Day-17 Analytics Tasks

Q1:

Title: Railway_Guages
Questions:

Here are 5 well-structured, end-to-end scenarios (with preprocessing → analysis → visualization), balanced by difficulty as you requested:

(Railway Dataset)

🟢 Scenario 1: Basic Data Loading & Cleaning

You are given a CSV file containing railway gauge data.

👉 Tasks:

1. **Load the dataset into a Pandas DataFrame.**
2. **Display the first 5 rows and column names.**
3. **Check for missing values and replace them with `0`.**
4. **Convert all gauge columns (`Broad`, `Metre`, `Narrow`, `Total`) to numeric types.**

🟢 Scenario 2: Simple Visualization

You want a quick understanding of total railway track growth.

👉 Tasks:

1. **Extract `Year` and `Total` columns.**
2. **Plot a line graph showing Total tracks over years.**
3. **Add:**
    - **Title**
    - **X and Y labels**
4. **Identify whether the trend is increasing or decreasing.**

🟡 Scenario 3: Filtering + Bar Chart

You are asked to analyze modern railway expansion.

👉 Tasks:

1. **Filter the dataset for years after 2000.**
2. **Select `Broad Gauge`, `Metre Gauge`, and `Narrow Gauge`.**
3. **Plot a grouped bar chart comparing all three gauges.**
4. **Add legend and proper labels.**
5. **Identify which gauge dominates in recent years.**

🟡 Scenario 4: Feature Engineering + Pie Chart

You want to analyze the contribution of each gauge type.

👉 Tasks:

1. **Calculate total sum of each gauge across all years.**
2. **Create a new structure (Series/DataFrame) for totals.**
3. **Plot a pie chart showing percentage contribution.**
4. **Add percentage labels (`autopct`).**
5. **Interpret which gauge contributes the most.**

🔴 Scenario 5: Advanced Analysis + Multiple Graphs

You are asked to perform a complete analysis of railway trends.

👉 Tasks:

1. **Create new columns:**
   - `% Broad Gauge`
   - `% Metre Gauge`
   - `% Narrow Gauge`
2. **Use NumPy (`np.diff`) to calculate yearly growth of Total tracks.**
3. **Plot:**
   - **Line graph for all gauges**

    - **Stacked bar chart showing composition**
4. **Highlight:**
    - **Years with highest growth**
    - **Decline in any gauge**
5. **Provide a final conclusion:**
   👉 **“Is the railway system shifting towards a single dominant gauge?”**

🎯 Coverage

✔ Data Preprocessing
✔ Pandas Operations
✔ NumPy Calculations
✔ Visualization:

- **Line graph**
- **Bar chart**
- **Pie chart**
- **Stacked chart**

NOTE: Make sure to save all the graphs in a graphs folder before pushing it to github.

Q2:

Title: IGN
Questions:

📊 `ign.csv`

🟢 Scenario 1: Data Loading & Preprocessing

You are given the `ign.csv` dataset containing game reviews.

👉 Tasks:

1. **Load the dataset using Pandas.**
2. **Display:**
   - **First 5 rows (`head()`)**
   - **Last 5 rows (`tail()`)**
   - **Shape of dataset**
3. **Remove the unnecessary column:**
   - **`"Unnamed: 0"` (index column)**
4. **Check for missing values in:**
   - **`score`, `genre`, `platform`**
5. **Handle missing values:**
   - **Fill numeric column `score` with mean**
   - **Fill categorical column `genre` with mode**
6. **Ensure correct data types:**
   - **`score` → float**
   - **`release_year`, `release_month`, `release_day` → integer**

🟢 Scenario 2: Line Graph (Score Trend) + Save

You want to analyze how game scores change over time.

👉 Tasks:

1. **Group data by `release_year`.**
2. **Calculate average score per year using Pandas.**
3. **Convert results into NumPy arrays.**
4. **Plot a line graph:**
   - **X-axis → `release_year`**
   - **Y-axis → average `score`**
5. **Add:**
   - **Title: *"Average Game Score Over Years"***
   - **Axis labels**

6. **Save the graph:** plt.savefig("avg_score_trend.png")

🟡 Scenario 3: Filtering + Bar Chart + Save

You want to compare top platforms.

👉 **Tasks:**

1. **Filter dataset where:**
   - **`score > 7`**
2. **Count number of high-rated games per `platform`.**
3. **Select top 10 platforms using Pandas.**
4. **Convert data into NumPy arrays.**
5. **Plot a bar chart:**
   - **X-axis → `platform`**
   - **Y-axis → count of games**
6. **Rotate x-axis labels for readability.**

Save the graph: plt.savefig("top_platforms_bar.png")

🟡 Scenario 4: Aggregation + Pie Chart + Save

You want to analyze genre distribution.

👉 **Tasks:**

1. **Count the number of games per `genre`.**
2. **Select top 5 genres using Pandas.**
3. **Prepare labels and values.**
4. **Plot a pie chart:**
   - **Labels → `genre`**

    - **Values → count**
5. **Add percentage labels (`autopct`).**

Save the graph: plt.savefig("genre_distribution.png")

🔴 Scenario 5: Advanced Analysis + Multiple Graphs

You are asked to perform a detailed analysis of review patterns.

👉 Part 1: Feature Engineering

1. **Create a new column:**
    - `score_category`:
        - `score >= 9` → **"Excellent"**
        - `7 <= score < 9` → **"Good"**
        - `< 7` → **"Average"**
2. **Convert `editors_choice`:**
    - `Y → 1, N → 0`

👉 Part 2: NumPy Analysis

3. **Use NumPy to:**
    - **Calculate yearly score growth using `np.diff()` on average yearly scores**

👉 Part 3: Visualizations

📈 Line Graph

4. **Plot trend of:**
    - **Average `score` per `release_year`**

📊 Stacked Bar Chart

5. **Show count of:**
    - `score_category` **per** `release_year`

📉 Histogram

6. **Plot distribution of:**
    - `score`

👉 Part 4: Save All Graphs

```
plt.savefig("score_trend.png")
plt.savefig("score_category_stacked.png")
plt.savefig("score_distribution.png")
```

👉 Part 5: Insights

Identify:

- **Which years had highest scores**
- **Whether high scores increased over time**
- **If `editors_choice` correlates with high scores**

🎯 Coverage

✔ Pandas:

- Cleaning
- Grouping
- Filtering
- Aggregation

✔ NumPy:

- Array conversion
- `np.diff()`

✔ Matplotlib:

- Line chart
- Bar chart
- Pie chart
- Histogram
- Saving plots

Q3:

Title: Scottish Hills Dataset
Questions:

📊 Scottish Hills Dataset

🟢 Scenario 1: Data Loading & Basic Cleaning

👉 **Tasks:**

1. **Load the dataset using Pandas.**
2. **Display:**
   - **First 5 rows**
   - **Column names**
3. **Check for missing values in:**
   - `Height`
   - `Region`
4. **Fill missing values:**
   - `Height` → **use mean**
   - `Region` → **use mode**
5. **Convert** `Height` **column to numeric if required.**

🟢 Scenario 2: Line Graph + Save

You want to see how heights vary.

👉 **Tasks:**

1. **Select:**
   - `Hill Name`
   - `Height`
2. **Take first 10 rows only.**
3. **Convert** `Height` **into a NumPy array.**
4. **Plot a line graph:**
   - **X-axis → index (0–9)**
   - **Y-axis →** `Height`

5. **Add title and labels.**

Save the graph: plt.savefig("hill_heights_line.png")

🟡 Scenario 3: Filtering + Bar Chart + Save

You want to analyze tall hills.

👉 **Tasks:**

1. **Filter hills where:**
   - `Height > 900`
2. **Count number of hills per `Region`.**
3. **Select top regions.**
4. **Convert results to NumPy arrays.**
5. **Plot a bar chart:**
   - **X-axis → `Region`**
   - **Y-axis → count**
6. **Rotate labels for clarity.**

Save graph: plt.savefig("tall_hills_bar.png")

🟡 Scenario 4: Pie Chart (Region Distribution) + Save

You want to see the distribution of hills.

👉 **Tasks:**

1. **Count the number of hills per `Region`.**
2. **Select top 5 regions.**
3. **Prepare labels and values.**
4. **Plot a pie chart.**
5. **Add percentage labels.**

Save graph: plt.savefig("region_distribution.png")

🔴 Scenario 5: Advanced Analysis + Multiple Graphs

You want deeper insights.

👉 Part 1: Feature Creation

1. Create a new column:
    - `Height Category`:
        - `Height >= 1000` → "Very High"
        - `800–999` → "High"
        - `< 800` → "Moderate"

👉 Part 2: NumPy Usage

2. Convert `Height` column to NumPy array.

Calculate height differences using:
np.diff()

👉 Part 3: Visualizations

📈 Line Graph

4. Plot height trend for all hills.

📊 Stacked Bar Chart

5. Show count of `Height Category` per `Region`.

📉 Histogram

6. Plot distribution of `Height`.

👉 Part 4: Save All Graphs

plt.savefig("height_trend.png")
plt.savefig("height_category_stacked.png")
plt.savefig("height_histogram.png")

👉 Part 5: Insights

- **Which region has tallest hills**
- **Which category is most common**
- **Distribution pattern of heights**

🎯 Simplification Done

✔ **Smaller subsets (first 10 rows)**
✔ **Simple conditions (`Height > 900`)**
✔ **Basic grouping**
✔ **Clear column usage**

Q4:

Title: House Sales Dataset
Questions:

📊 House Sales (`kc_house_data.csv`)

🟢 Scenario 1: Data Loading & Basic Cleaning

👉 **Tasks:**

1. Load the dataset using **Pandas**.
2. Display:
   - First 5 rows
   - Column names
3. Check for missing values in:
   - `bedrooms`
   - `bathrooms`
   - `sqft_living`
   - `price`
4. Fill missing values:
   - `bedrooms` → use **mode**
   - `bathrooms` → use **mean**
   - `sqft_living` → use **mean**
   - `price` → use **mean**
5. Convert these columns to numeric if required:
   - `bedrooms`
   - `bathrooms`
   - `sqft_living`
   - `price`

🟢 Scenario 2: Line Graph + Save

👉 **Tasks:**

1. Select:
    - `id`
    - `price`
2. Take the first **10 rows only**.
3. Convert `price` into a **NumPy array**.
4. Plot a **line graph**:
    - X-axis → index (0–9)
    - Y-axis → Price
5. Add:
    - Title
    - X-label
    - Y-label
6. Save the graph: plt.savefig("house_prices_line.png")

🟡 Scenario 3: Filtering + Bar Chart + Save

🎯 **You want to analyze expensive houses.**

👉 **Tasks:**

1. Filter houses where:
    - `price > 1000000`
2. Count number of houses per:
    - `bedrooms`
3. Select top bedroom categories.
4. Convert results to **NumPy arrays**.
5. Plot a **bar chart**:
    - X-axis → Bedrooms
    - Y-axis → Count
6. Rotate labels if needed.
7. Save graph: plt.savefig("expensive_houses_bar.png")

🟡 Scenario 4: Pie Chart (Bedroom Distribution) + Save

🎯 **You want to see the distribution of house types.**

👉 **Tasks:**

1. Count number of houses by:
    - `bedrooms`
2. Select **top 5 bedroom categories**.
3. Prepare:
    - Labels
    - Values
4. Plot a **pie chart**.
5. Add percentage labels.
6. Save graph: plt.savefig("bedroom_distribution.png")

🔴 Scenario 5: Advanced Analysis + Multiple Graphs

🎯 **You want deeper insights from the house dataset.**

👉 Part 1: Feature Creation

Create a new column:

Price Category

- `price >= 1000000` → `"Luxury"`
- `500000 – 999999` → `"Mid Range"`
- `< 500000` → `"Affordable"`

👉 Part 2: NumPy Usage

1. Convert `price` column to a **NumPy array**.
2. Calculate price differences using:

np.diff()

👉 Part 3: Visualizations

📈 Line Graph

Plot **price trend for all houses**.

📊 Stacked Bar Chart

Show count of **Price Category per Bedrooms**.

📉 Histogram

Plot distribution of:

- `price`

👉 Part 4: Save All Graphs

plt.savefig("price_trend.png")
plt.savefig("price_category_stacked.png")
plt.savefig("price_histogram.png")

👉 Part 5: Insights

Answer these:

1. Which **bedroom category** has the most expensive houses?
2. Which **price category** is most common?
3. What is the **distribution pattern of house prices**?
    - Right-skewed?
    - Normal?
    - Concentrated in a lower price range?

🎯 Simplification Done

✔ Smaller subsets (first 10 rows)
✔ Simple filtering (`price > 1000000`)
✔ Basic grouping (`bedrooms`)
✔ Clear and easy columns from your dataset
✔ Similar style to your Scottish Hills task

Q5:

Title: Car Data Dataset
Questions:

🚗 Car Data Dataset (`cardata.csv`)

🟢 Scenario 1: Data Loading & Basic Cleaning

Understand the dataset structure and prepare it for analysis.

👉 **Tasks:**

- **Load the dataset using Pandas.**
- **Display:**
  - **First 5 rows**
  - **Last 5 rows**
  - **Column names**
  - **Shape of dataset**
- **Check data types of all columns.**
- **Check for missing values in:**
  - `Selling_Price`
  - `Present_Price`
  - `Kms_Driven`
  - `Fuel_Type`
- **Fill missing values:**
  - `Selling_Price` → **mean**
  - `Present_Price` → **mean**
  - `Kms_Driven` → **mean**
  - `Fuel_Type` → **mode**
- **Convert numeric columns to proper numeric type if required:**
  - `Selling_Price`
  - `Present_Price`
  - `Kms_Driven`
  - `Year`
- **Convert `Selling_Price` and `Kms_Driven` into NumPy arrays.**
- **Use NumPy to calculate:**
  - **minimum selling price**

    - maximum selling price
    - average selling price

🟢 Scenario 2: Selling Price Trend (Line Graph)

See how selling prices vary for a small sample of cars.

👉 Tasks:

- **Select:**
    - `Car_Name`
    - `Selling_Price`
- **Take the first 10 rows only using Pandas.**
- **Convert `Selling_Price` into a NumPy array.**
- **Plot a line graph using Matplotlib:**
    - **X-axis → row index (0–9)**
    - **Y-axis → Selling Price**
- **Add:**
    - **title**
    - **x-axis label**
    - **y-axis label**
    - **markers**
- **Save the graph with a suitable filename.**

🟡 Scenario 3: Expensive Cars Analysis (Filtering + Bar)

Find which fuel types are most common among expensive cars.

👉 Tasks:

- **Filter cars where:**
    - `Selling_Price > 10`
- **Group the filtered data by:**
    - `Fuel_Type`
- **Count number of cars in each fuel type.**
- **Convert:**
    - **fuel type labels**

    - counts
      into NumPy arrays.
- Plot a bar chart using Matplotlib:
    - X-axis → Fuel Type
    - Y-axis → Count of expensive cars
- Add:
    - title
    - x-label
    - y-label
- Save the graph.

🟡 Scenario 4: Fuel Type Distribution (Pie Chart)

Understand the overall distribution of cars by fuel type.

👉 **Tasks:**

- Count the number of cars in each:
    - `Fuel_Type`
- Select all categories or top categories if needed.
- Prepare:
    - labels
    - values
- Convert values into a NumPy array.
- Plot a pie chart using Matplotlib.
- Add:
    - percentage labels
    - title
- Save the graph.

🟡 Scenario 5: Present Price vs Selling Price (Scatter Plot)

Check whether cars with higher present price also have higher selling price.

👉 **Tasks:**

- Select:
    - `Present_Price`
    - `Selling_Price`

- Remove missing values if any.
- Take a smaller sample (for example first 50 or 100 rows) using Pandas.
- Convert both columns into NumPy arrays.
- Plot a scatter plot using Matplotlib:
  - X-axis → `Present_Price`
  - Y-axis → `Selling_Price`
- Add:
  - title
  - x-label
  - y-label
- Observe whether there is a positive relationship.
- Save the graph.

🔴 Scenario 6: Car Age Category Analysis + Bar Chart

Create a new feature using year and compare car categories.

👉 **Tasks:**

- Create a new column using Pandas:

Car Age Category

- `Year >= 2015` → `"New"`
- `2010 to 2014` → `"Medium"`
- `< 2010` → `"Old"`
- Count number of cars in each:
  - `Car Age Category`
- Convert category names and counts into NumPy arrays.
- Plot a bar chart using Matplotlib:
  - X-axis → Car Age Category
  - Y-axis → Count
- Add title and labels.
- Save the graph.

🔴 Scenario 7: Kms Driven Distribution (Histogram)

Understand how the cars are distributed based on kilometers driven.

👉 **Tasks:**

- **Select:**
  - `Kms_Driven`
- **Convert it into a NumPy array.**
- **Plot a histogram using Matplotlib:**
  - **X-axis → Kms Driven**
  - **Y-axis → Frequency**
- **Choose suitable number of bins.**
- **Add:**
  - **title**
  - **x-label**
  - **y-label**
- **Save the graph.**
- **Observe whether most cars have lower or higher mileage.**

🔴 Scenario 8: Transmission-wise Selling Price Comparison

Compare average selling price for manual vs automatic cars.

👉 Tasks:

- **Group data by:**
  - `Transmission`
- **Calculate:**
  - **average** `Selling_Price`
- **Convert transmission labels and average prices into NumPy arrays.**
- **Plot a bar chart using Matplotlib:**
  - **X-axis → Transmission**
  - **Y-axis → Average Selling Price**
- **Add title and labels.**
- **Save the graph.**

🔴 Scenario 9: Seller Type Analysis

Compare how many cars are sold by dealers and individuals.

👉 **Tasks:**

- Count number of cars by:
  - `Seller_Type`
- Convert results into NumPy arrays.
- Plot a bar chart or pie chart using Matplotlib.
- Add labels and title.
- Save the graph.
- Identify which seller type is more common.

🔴 Scenario 10: Advanced Analysis + Multiple Graphs

Perform deeper analysis using Pandas, NumPy, and Matplotlib.

👉 Part 1: Feature Creation

Create a new column:

Price Difference

- `Price Difference = Present_Price - Selling_Price`

This shows how much value the car has depreciated.

👉 Part 2: NumPy Usage

- Convert `Selling_Price` into a NumPy array.
- Use NumPy to calculate price changes between consecutive rows using:
  - `np.diff()`
- Convert `Price Difference` column into a NumPy array.
- Find:
  - average depreciation
  - maximum depreciation
  - minimum depreciation

👉 Part 3: Visualizations

📈 **Line Graph**

- **Plot Selling_Price trend for all cars.**

📊 **Bar Chart**

- **Show average Selling_Price by Fuel_Type.**

📉 **Histogram**

- **Plot distribution of Selling_Price.**

👉 **Part 4: Insights**

Answer these:

- **Which fuel type has the highest average selling price?**
- **Which transmission type has higher average selling price?**
- **Are most cars concentrated in lower selling prices or higher selling prices?**
- **Do older cars tend to have lower selling prices?**

🎯 Best Simple & Correct Columns to Use in This Dataset

These are the best columns for your tasks:

Numeric Columns

- `Selling_Price`
- `Present_Price`
- `Kms_Driven`
- `Year`
- `Owner`

Categorical Columns

- `Fuel_Type`
- `Seller_Type`
- `Transmission`
- `Car_Name`